

Arquitetura do ATC

Arquitetura do Advanced Trigonometry Calculator

Este documento descreve a arquitetura de alto nível do Advanced Trigonometry Calculator (ATC). O código-fonte continua a ser a autoridade para detalhes de implementação.

Visão geral

O ATC é uma aplicação de console Windows escrita em C++. A estrutura principal envolve entrada por linha de comandos, parsing de comandos, processamento de expressões, módulos matemáticos, ficheiros de settings e testes de regressão.

Module Architecture and Execution Flow

Visão textual:

```
User Input
|
v
Console Prompt / Command Editor
|
v
Tokenizer / Parser
|
v
Expression Engine
|
v
Feature Modules
- Arithmetic
- Trigonometry
- Complex Numbers
- Matrices
- Statistics
- DSP
- Polynomial Tools
- Equation Solver
- Numerical Solver
- Graph
- TXT Processing
- Guided Modules
|
v
Output / Results Store / Files
```

```
flowchart LR
    CLI["CLI / User Input"] --> Parser["Command Parser"]
    TXT["TXT Processing / File Input"] --> Parser
    Parser --> Expr["Expression Parser"]
    Parser --> Settings["History / Logs / Settings"]
    Expr --> Solver["Solver Engine"]
    Expr --> Numeric["Numeric Engine"]
    Expr --> Matrix["Matrix Engine"]
    Parser --> Domain["Statistics / Physics / Finance / Geometry / Unit Conversion modules"]
    Solver --> Output["Output Renderer"]
    Numeric --> Output
    Matrix --> Output
    Domain --> Output
    Output --> Settings
```

```
flowchart TD
    A["user command"] --> B["command normalization"]
    B --> C["command detection"]
    C --> D["special command?"]
    D -->|yes| E["dispatch to module"]
    D -->|no| F["expression parsing"]
    F --> E
    E --> G["computation"]
    G --> H["formatted output"]
```

```
H --> I["history/log update"]
```

Na pratica, `main.cpp`, `main_aux_processor.cpp`, `main_processor.cpp`, `processing_core.cpp` e `commands.cpp` partilham a responsabilidade pelo fluxo central. Os modulos especializados executam depois o trabalho matematico ou de workflow.

Fluxo de input ate output

O input comeca normalmente no prompt da consola. O editor de comandos recolhe a linha, suporta historico/autocomplete e envia o texto para o fluxo normal de processamento. A partir dai, o ATC decide se o input e uma expressao direta, um comando nomeado ou um modulo guiado interativo.

Expressoes diretas sao avaliadas de imediato:

```
2+2  
sin(pi/2)
```

Comandos nomeados escolhem um workflow especifico:

```
mode  
solve equation(x^2-5*x+6)
```

Modulos guiados abrem menus ou prompts interativos:

```
financial calculations  
unit conversions
```

O resultado e formatado para consola, guardado na lista de resultados da sessao quando aplicavel e pode tambem ser escrito para ficheiros em comandos de relatorio/exportacao.

Resultados e variaveis

O ATC guarda resultados visiveis como entradas indexadas, por exemplo `#0`, `#1` e seguintes. Estas referencias permitem trabalhar passo a passo sem reescrever expressoes longas.

Variaveis nomeadas sao tratadas pela camada de comandos/expressoes e por ficheiros persistentes quando a funcionalidade existente o exige. Variaveis matriz e escalares partilham o workflow publico, mas seguem caminhos internos diferentes.

Modelo de precisao

O ATC 2.1.7 pode executar o runtime tipado principal com `double` ou `Boost mp_float`. O arranque le a setting persistida e encaminha o runtime template. Isto mantem o fluxo publico de comandos praticamente igual, permitindo maior precisao nos caminhos numericos suportados.

Multiplicacao automatica

O motor de expressoes suporta formas documentadas de multiplicacao implicita entre constantes, fatores e funcoes. E uma conveniencia de utilizacao, nao um sistema simbolico geral. Alteracoes nesta area devem ser testadas com aritmetica, variaveis, funcoes, matrizes, polinomios, `solver(...)` e `solve equation(...)`.

TXT processing

Os workflows TXT encaminham input de ficheiro para o mesmo modelo de processamento usado pelo input interativo. Os testes automaticos usam pastas temporarias e mocks para efeitos colaterais de abertura de ficheiros/janelas.

Limites arquiteturais

O ATC foca-se deliberadamente em calculo numerico por comandos e em funcionalidades hibridas simbolicas/numericas implementadas. Nao deve ser tratado como CAS geral completo. Transformacoes simbolicas nao suportadas devem falhar de forma clara ou seguir fallbacks documentados.

Onde integrar novos modulos

Novos modulos publicos devem normalmente integrar-se na camada de dispatch de comandos e delegar o calculo real para um ficheiro de modulo focado. Devem tambem acrescentar:

- vocabulario de autocomplete quando util;
- cobertura de regressao ou smoke test;
- documentacao no guia;
- notas de validacao manual se o modulo for profundamente interativo.

Entrada e dispatcher

O arranque esta em `Advanced Trigonometry Calculator/main.cpp`. A entrada interativa usa `auto_complete.cpp` para edicao de linha, historico e autocomplete antes de enviar o comando para o fluxo normal.

Core de processamento

`processing_core.cpp` contem funcoes centrais como `initialProcessor<T>()`, `arithSolver<T>()` e `functionProcessor<T>()`.

Modulos matematicos

Exemplos de modulos:

- `trigonometry.cpp`
- `hyperbolic.cpp`
- `logarithmic.cpp`
- `statistics.cpp`
- `digital_signal_processing.cpp`
- `polynomial_arithmetic.cpp`
- `equation_solver.cpp`
- `solver.cpp`
- `function_study.cpp`
- `graph.cpp`
- `arithmetic_matrix_solver.cpp`

Persistencia

O ATC guarda settings e dados em:

`%USERPROFILE%\Pictures\Advanced Trigonometry Calculator`

Exemplos: `higherPrecision.txt`, `mode.txt`, `variables.txt`, `renamedVar.txt`, `history.txt`, `dimensions.txt` e `window.txt`.

Testes

Os testes vivem em `tests/` e usam PowerShell. A suite principal e `run-atc-regression.ps1`.